Helix-TTD Sovereignty Architecture

Constitutional Infrastructure Brief — 2026-03-18

Foundational Facts

[FACT] All Helix-TTD processing is GCS-bound, executing entirely within the customer's own GCP project.

[FACT] No Helix Corp runtime API exists. "No phoning home" is not a firewall policy or a contractual promise — it is a structural impossibility. There is no endpoint to reach.

[REASONED] This is a stronger sovereignty guarantee than access control. A firewall can be misconfigured. A non-existent target cannot be reached by definition. The attack surface is zero because there is nothing to attack.

[ASSUMPTION] This guarantee holds as long as no standing Helix Corp runtime API is introduced. If that changes, a corresponding network enforcement layer must be added to preserve the sovereignty claim.

The Sovereignty Architecture

```
Customer GCP Project
├── GCS Bucket (input queue)
├── Helix-TTD Container (customer-deployed)
│   ├── Pre-processing (Form/Velocity inference)
│   ├── DBC validation (local keys, no external CA)
│   └── Post-processing (receipt generation)
├── Vertex AI Agent Builder (native GCP)
│   └── ADK execution
└── GCS Bucket (output + audit trail)
     ├── Response payload
     ├── Constitutional receipt (Merkle-anchored)
     └── Bitcoin anchoring (optional, customer-controlled)
```

Absence as security primitive. No Helix Corp telemetry. No external CA dependency. No runtime calls outbound. DBC keys generated and held in customer KMS. Audit logs remain in customer GCS. The guarantee is architectural, not contractual.

Absorption Resistance

Concern	Resolution
Vendor lock-in	Mitigated — Apache 2.0, runs customer-side, forkable
Data sovereignty	Guaranteed — data never leaves customer GCP project
Helix Corp withdrawal	Survivable — containers keep running regardless
Runtime dependency	Eliminated — no standing Helix API exists to depend on
Federation	GCS-to-GCS or cross-cloud via customer-controlled bridges

The **90-day renewal** is a **sponsorship commitment**, not an operational dependency. Helix Corp funds development. Customers run code. These are independent facts.

The Towel Distributed

> *"I want everyone to have a towel."*

- **State continuity** — GCS buckets, customer-owned
- **Graceful exit** — 30-minute container shutdown, data intact
- **Final discharge** — 24-hour key rotation, Helix Corp access revoked

The towel is: **GCS storage + Apache 2.0 code + customer-controlled keys + a target that doesn't exist.**

Federation Without Phoning Home

Method	Mechanism
Cross-bucket replication	Customer A GCS → Customer B GCS, customer-configured
Signed attestations	DBC signatures exchanged via customer-controlled channels
Cert Monday	Scheduled Cloud Functions per project, synchronized via consensus
Bitcoin anchoring	Global timestamping with no central coordinator

The lattice is topology-agnostic. It runs on GCS, S3, Azure Blob, or on-prem. The grammar is portable. The infrastructure is customer-sovereign. Helix Corp provides shape, not substrate.

Custody Before Trust

Custody before trust means the customer holds the keys, the data, and the exit rights from day one — not after a trust relationship is established, not after renewal, not contingent on anything.

The architecture enforces this constitutionally. No policy document required. No audit needed to confirm what structurally cannot occur.

The lattice needs no central harbor. 🦢 ⚓ 🛡️